Parallel Gaussian Splatting via Reduction Trees and Hardware-Aware Optimization

Your Name

Abstract

3D Gaussian Splatting has emerged as an efficient and high-quality alternative to neural radiance fields, but its rendering and optimization pipelines remain constrained by inherently sequential operations and redundant computation. In particular, alpha blending is typically performed in a front-to-back manner, limiting parallelism, while optimization and rendering repeatedly process spatially correlated primitives with limited reuse.

In this paper, we present a GPU acceleration framework for Gaussian Splatting that restructures both rendering and optimization to exploit fine-grained parallelism on modern GPUs. We introduce a reduction-tree formulation of alpha blending that transforms the traditionally sequential compositing process into a parallel operation, enabling efficient multi-threaded execution across GPU cores. Building on this formulation, we map Gaussian splatting to the Thor GPU architecture using cooperative threading to maximize occupancy and memory throughput.

To further accelerate training, we fuse the Adam optimizer directly into the Gaussian update pipeline, reducing memory traffic and synchronization overhead. We also propose an enhanced Gaussian primitive augmented with surface normals, enabling directional appearance modeling and reducing unnecessary shading computations. Finally, we exploit spatial locality and geometric similarity by pruning redundant evaluations: neighboring points influenced by the same Gaussian are skipped, and rendering is performed only for point subsets whose geometric contribution exceeds a predefined threshold.

Together, these techniques significantly reduce computation and improve scalability without sacrificing visual quality. Our results demonstrate that restructuring Gaussian splatting around parallel reduction, optimization fusion, and spatial locality yields substantial performance gains, positioning Gaussian Splatting as a practical foundation for real-time and large-scale rendering on specialized GPU architectures.

1 Introduction

Gaussian Splatting enables real-time novel view synthesis by representing scenes with explicit anisotropic Gaussian primitives. Despite its efficiency, key stages of the pipeline—most notably alpha blending and parameter optimization—remain inherently sequential and redundant. This paper addresses these limitations by reformulating Gaussian splatting around parallel compositing and hardware-aware execution.

2 Background: Gaussian Splatting

We briefly review the Gaussian Splatting pipeline, including Gaussian projection, visibility determination, opacity evaluation, and front-to-back alpha blending. While this formulation achieves high visual quality, the compositing step introduces prefix-style dependencies that limit parallel execution.

3 Related Work

Gaussian Splatting and rendering acceleration. The original 3D Gaussian Splatting formulation is evaluated on Blender synthetic scenes (e.g., Lego, Chair, Hotdog), Mip-NeRF 360 indoor and outdoor scenes, and Tanks and Temples datasets. Subsequent works such as FlashGS and Speedy-Splat optimize visibility, kernel scheduling, and pruning, reporting up to $7.2\times$ and $6.71\times$ rendering speedups respectively on these benchmarks.

Architecture-aware analysis. GSCore (ASPLOS 2024) characterizes Gaussian Splatting on GPU architectures using Blender and real-world datasets, identifying rasterization and alpha blending as dominant bottlenecks while retaining sequential compositing.

Training and primitive extensions. Other works accelerate training via distributed optimization or enhance primitives with geometry and semantic cues, often evaluated on indoor scans, autonomous driving datasets, and large-scale outdoor scenes. These approaches motivate richer primitives but increase computational pressure on the rendering pipeline.

Summary. Across common datasets such as Blender, Mip-NeRF 360, and Tanks and Temples, prior work achieves substantial speedups through culling and kernel optimizations. However, the core alpha blending dependency remains sequential. Our work directly targets this limitation via parallel reduction.

4 Parallel Alpha Blending via Reduction Trees

We introduce a reduction-tree formulation of alpha blending that converts sequential transmittance accumulation into a parallel reduction. This enables compositing to be executed in logarithmic depth using GPU thread cooperation, while preserving numerical correctness and visual fidelity.

5 Hardware-Aware GPU Mapping on Thor

We map the reduction-tree compositing and Gaussian evaluation to the Thor GPU architecture. Our design leverages cooperative threading, memory coalescing, and shared-memory reductions to maximize utilization and throughput.

6 Optimization Fusion

We fuse the Adam optimizer into the Gaussian update pipeline to reduce memory traffic and synchronization overhead. This fusion enables faster convergence and improves training throughput without altering optimization behavior.

7 Enhanced Gaussian Primitive with Normals

We augment Gaussian primitives with surface normals to enable directional appearance modeling. This enhancement

improves shading fidelity and reduces redundant computations by avoiding unnecessary evaluation on the occluded side of a primitive.

8 Spatial Locality and Redundancy Pruning

We exploit spatial locality by skipping redundant evaluations for neighboring points influenced by identical or near-identical Gaussians. Additionally, we apply threshold-based pruning to avoid rendering point subsets whose contribution does not significantly change.

9 Experiments

10 Evaluation

10.1 Datasets

We evaluate on Blender synthetic scenes (Lego, Chair, Hotdog), Mip-NeRF 360 indoor and outdoor scenes, and Tanks and Temples. These datasets are standard across recent Gaussian Splatting literature and enable direct comparison.

10.2 Baselines

We compare against vanilla Gaussian Splatting, FlashGS, and Speedy-Splat using their reported configurations and metrics.

10.3 Metrics

We report average and minimum FPS, frame time breakdowns, PSNR, SSIM, LPIPS, and memory traffic statistics.

10.4 Results and Ablations

We evaluate the contribution of reduction-tree alpha blending, GPU thread mapping, optimizer fusion, spatial pruning, and normal-aware primitives through controlled ablation studies.

11 Discussion and Limitations

We discuss numerical stability, ordering sensitivity, and compatibility with existing early-termination heuristics, as well as limitations related to extreme depth complexity.

12 Conclusion

We present a parallel, hardware-aware Gaussian Splatting framework that restructures alpha blending, optimization, and primitive evaluation. Our results show that reduction-based compositing combined with locality-aware execution significantly improves performance and scalability.

References